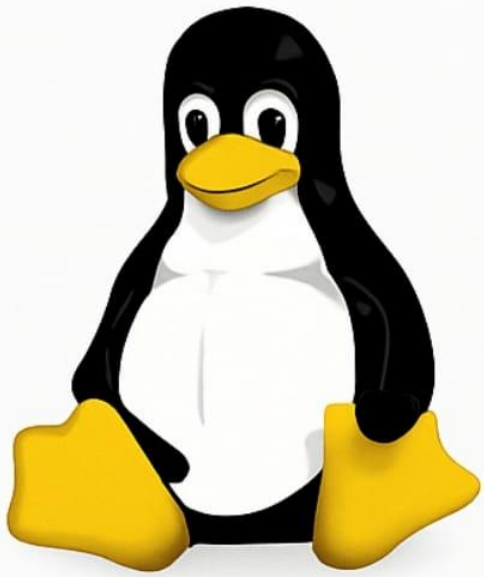
Complete Linux Theory

for DevOps Beginners



Swipe

1. Introduction

In today's fast-paced software industry, DevOps has become a core methodology for building, deploying, and managing modern applications. At the foundation of this entire ecosystem lies **Linux**, the open-source operating system that powers servers, cloud platforms, automation tools, and container technologies.

Linux was created in 1991 by **Linus Torvalds**, a computer science student from Finland. What started as a small personal project quickly evolved into one of the most powerful and widely used operating systems in the world, thanks to contributions from millions of developers globally.

For modern DevOps engineers, understanding Linux is not optional it is a core fundamental skill. Every cloud server, container engine, CI/CD system, and automation pipeline relies on Linux's architecture and command-line environment.

2. What is Linux and Why is It Important for DevOps?

Linux is an open-source operating system built on Unix design principles. Over the years, it has evolved into the default platform for modern software engineering.

Linux powers:

- Server infrastructure (web, database, application servers)
- Cloud platforms (AWS, Azure, GCP, IBM Cloud)
- Container engines (Docker)
- CI/CD pipelines (GitHub Actions, Jenkins, GitLab CI)
- Cybersecurity & networking systems
- Developer environments & automation tools

Linux is not just an OS it is the ecosystem on which DevOps workflows naturally fit.

▧ 3. Why Linux Is Essential in DevOps

1 Open-Source Nature

Linux gives DevOps engineers complete flexibility to:

- Customize the OS
- Optimize for servers/containers
- Understand internal system behavior
- Automate without restrictions

Open-source = freedom + transparency + low cost

---

2 Stability & Reliability

Linux can run for months or years without reboot, making it ideal for:

- Production servers
- Cloud workloads
- Microservices
- High-availability systems

Its strong process and memory management keeps services stable under load.

---

3 Command-Line Interface

DevOps = automation, and that begins with the CLI.

Engineers use Linux CLI to:

- Write shell scripts
- Deploy applications
- Manage servers
- Configure cloud instances

- View logs and processes
- Work with containers & Kubernetes

Almost every cloud platform expects CLI skills.

4️⃣ Massive DevOps Tool Ecosystem

Linux supports all major DevOps tools:

- Docker
- Kubernetes, K3s, OpenShift
- Ansible, Puppet, Chef
- Jenkins, GitHub Actions, GitLab CI
- Git
- Prometheus, Grafana, ELK

These rely on Linux kernel features like **cgroups** and **namespaces**, making Linux their natural environment.

5️⃣ Portability & Cloud-Native Compatibility

Linux runs on:

- Cloud VMs
- On-prem servers
- Containers
- Embedded & edge devices

This ensures consistent behavior across:

- Development
- Testing
- Production
- Cloud environments

This consistency is a major reason Linux dominates DevOps.

▧ 4. Linux Philosophy – "Everything is a file"

Linux follows a fundamental design principle known as the **Unix Philosophy**, where:

"Almost everything in Linux is represented as a file."

This means Linux treats hardware, system processes, and kernel information as files located within the file system.

Examples:

- **CPU Information** → /proc/cpuinfo
- **Memory Usage** → /proc/meminfo
- **Running Processes** → /proc/<PID>
- **Hard Disk Devices** → /dev/sda
- **Network Sockets & Pipes** → Appears as file-like objects

Why This Matters in DevOps:

- Enables powerful automation using shell scripts
- Makes system monitoring easier (just read a file to get info)
- Allows tools like Docker, Kubernetes, Prometheus, etc., to interact efficiently with the OS
- Makes Linux highly flexible and transparent

This philosophy is one of the main reasons Linux is the preferred OS for **servers, cloud environments, and DevOps workloads**.

5. What is a Kernel?

The kernel is the core part of an operating system.
It manages memory, processes, hardware, file systems, and system calls — acting as the bridge between software and hardware.

Difference Between Monolithic Kernel & Microkernel

Feature	Monolithic Kernel (Linux)	Microkernel
Structure	All OS services run in kernel space	Only minimal core in kernel; rest in user space
Performance	Very fast, low overhead	Slower due to message passing
Security	Moderate (large kernel)	High (small kernel surface)
Stability	Fast & stable, but crashes affect whole system	Highly stable; failures isolated
Communication	Direct function calls	IPC (inter-process communication)
Complexity	Large, complex kernel	Smaller, simpler core
Memory Usage	Higher	Lower
Examples	Linux, Unix	Minix, QNX, Hurd
Best For	Servers, Cloud, DevOps, Containers	Embedded & mission-critical systems

Note: Which Kernel Does Linux Use?

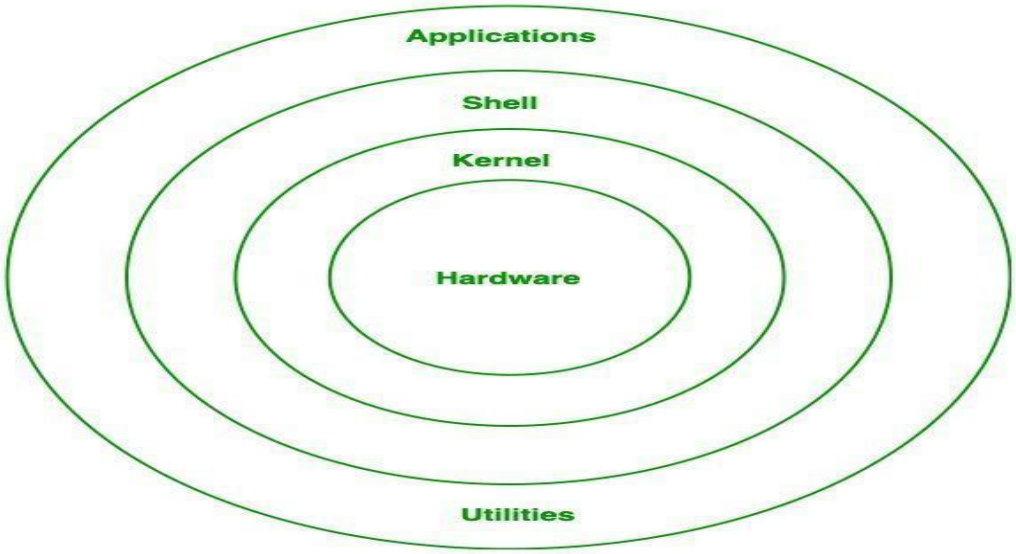
✓ **Linux uses a *Monolithic Kernel*.**

Why?

- It provides **faster performance**
- Handles **heavy workloads** efficiently
- Works best with **containers, cloud platforms, and servers**
- Offers predictable behavior under load

This design is one major reason Linux dominates DevOps, server infrastructure, and cloud computing.

6. Linux Architecture



Linux Architecture Explanation of the Diagram

This diagram shows **five major layers** of the Linux operating system:

Hardware (Innermost Layer)

This is the **core physical layer**.
It includes:

- CPU
- RAM
- Hard disk / SSD
- Keyboard, mouse, display
- Network cards
- Other I/O devices

Role:
Provides the raw computing resources. But hardware cannot understand high-level instructions, and it cannot talk to applications directly. It needs the kernel to manage everything.

Kernel (Second Layer)

The kernel sits directly on top of the hardware.
It is the *heart* of Linux.

What it does:

- Controls memory allocation
- Manages running processes
- Handles device drivers
- Manages file systems
- Provides networking
- Executes system calls (requests from programs)

In simple words:
The kernel is like a manager who talks to hardware on behalf of applications.
Without the kernel, hardware is silent, and software is helpless.

Shell (Third Layer)

The shell is the **command interpreter**.
You can picture it as the "middleman" between the user and the kernel.

Examples of shells:

- Bash
- Zsh
- Fish

Role:

- Takes commands typed by the user
- Converts them into instructions for the kernel
- Shows the results back to the user

Analogy:
If the kernel is the "engine," the shell is the "steering wheel."

Applications (Outer Layer)

Applications are the programs you install and use.

Examples:

- Chrome / Firefox
- VS Code
- Python scripts
- Docker
- Git

How they work:
They interact with the shell or directly with system libraries, which then communicate with the kernel.

Applications cannot directly access hardware — they always go through the lower layers.

Utilities (Outer Surrounding Layer)

Utilities are built-in system tools that come with Linux — the small but powerful command-line programs.

Examples:

- ls
- cp
- mkdir
- cat
- grep
- chmod

Role:

- Provide everyday system operations
- Help users interact with files, permissions, and processes
- Many utilities rely on the shell to execute and on the kernel for hardware access

These utilities form the backbone of system management.

How These Layers Work Together

A simple example explains everything:

When you type ls in the terminal:

1. You enter a command in the **Shell**
2. Shell passes the command to the **Kernel**
3. Kernel reads directory information from **Hardware (disk)**
4. Kernel sends the result back to the **Shell**
5. Shell displays output to the **Application (terminal)**

All five layers quietly work in harmony.

7. Linux Flavours

Linux distributions (distros) are different versions of Linux, each built for different needs — beginners, enterprises, DevOps, cybersecurity, or high-performance systems. Each distro family uses its own **package manager**, which installs, updates, and manages software.

Below is the **combined, final, well-structured version**:

A. Debian-Based Flavours

Known For: Stability, ease of use, large community.

Popular Distros:

• Ubuntu
• Linux Mint
• Kali Linux
• Pop!_OS
• Debian (original)

Package Manager:

• **APT (Advanced Package Tool)**
Common commands:

- apt update

- apt install <package>

Best For:

• Beginners
• DevOps
• Desktops
• Cloud servers

B. Red Hat–Based Flavours

Known For: Enterprise-grade performance, production-level security.

Popular Distros:

• RHEL (Red Hat Enterprise Linux)
• CentOS / Rocky Linux
• Fedora

Package Manager:

• **YUM (Yellowdog Updater Modified)**
• **DNF (Dandified YUM)** – newer version

Use Case:

• Enterprise servers
• Corporates
• DevOps pipelines

C. Arch-Based Flavours

Known For: Lightweight, customizable, rolling release.

Popular Distros:

• Arch Linux
• Manjaro

Package Manager:

• **pacman**

Best For:

• Advanced users
• Custom setups
• Learning Linux internals

🟦 D. SUSE-Based Flavours

Known For: Strong system management, enterprise stability.

Popular Distros:

• openSUSE
• SLES (SUSE Linux Enterprise Server)

Package Manager:

• **zypper**

Best For:

• Enterprise workloads
• SAP systems
• High availability servers

🟦 E. Security & Forensics Flavours

Known For: Penetration testing, forensics, cybersecurity.

Popular Distros:

• Kali Linux
• Parrot OS
• BackBox

Best For:

• Ethical hackers
• Cybersecurity learners
• Pentesting labs

▧ 8.User Types in Linux

Linux uses a permission-based security model, and different categories of users exist for safe system operation.

1 Root User (Superuser)

- Has complete control over the entire system
- Can install software, modify system files, add users, and change permissions
- Represented by: uid=0

2 Normal Users

- Regular system users with limited privileges
- Can access only their home directory and permitted commands
- Ideal for everyday operations and development tasks

3 System / Service Users

- Created automatically by system services
- Run background processes safely without needing full permissions
- Examples:
    - mysql→ MySQL Server
    - nginx→ Nginx Web Server

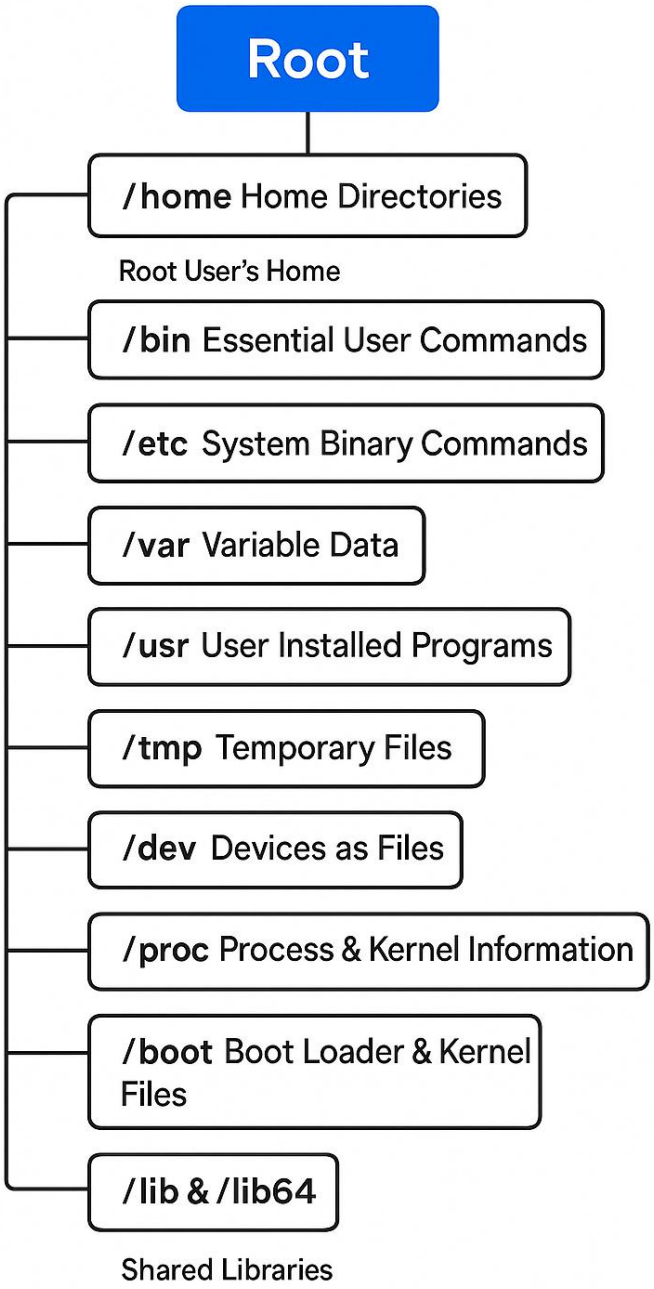
Why This Matters in DevOps

Understanding user types is essential for:

- Managing file permissions
- Running services safely
- Setting up automation tools
- Configuring CI/CD agents
- Deploying applications securely

9.Linux File System

The Linux File System is a hierarchical, tree-shaped structure where everything begins at the **root directory (/)**.
Linux has a **single directory tree**, and all files, folders, devices, and processes exist inside this structure.



1. Root Directory (/) – The Top Level

The **root directory** is the starting point of the entire Linux file system.
Every file and folder is located under /.

Example Structure:

/

|-- bin

|-- home

|-- etc

|-- var

---

2. Important Directories Under Root (/)

Below are the major directories and their purposes:

/home – User Home Directories

Contains all regular users' personal files.

Examples:

/home/shubham

/home/student

Used For:
• Documents
• Downloads
• Custom configurations
• Scripts

/root – Root User's Home

Home directory of the **superuser (root)**.
(Not the same as /.)

/bin – Essential User Commands

Contains important commands needed for system operation.

Examples:
• ls
• cp
• mv
• mkdir
• cat

Available even in recovery mode.

🗀 /sbin – System Binary Commands

Contains administrative/system commands used mostly by **root**.

Examples:
• reboot
• shutdown
• fdisk
• ifconfig

Used for system maintenance.

🗀 /etc – System Configuration Files

The **brain of the Linux system**.

Contains:
• Network configurations
• User accounts
• Service configurations
• Application settings
• Startup scripts

Examples:

/etc/passwd

/etc/hosts

/etc/fstab

/var – Variable Data

Stores files that change frequently.

Includes:
• Logs → /var/log
• Cache
• Temporary service data
• Mail spool

Important for **monitoring and DevOps logs**.

/usr – User Installed Programs

The **largest directory** in Linux.

Contains:
• Applications
• Libraries
• Documentation

Examples:

/usr/bin

/usr/lib

/usr/share

/tmp – Temporary Files

Temporary data used by:
• Applications
• Installers
• Running processes

Files may auto-delete after reboot.

/dev – Devices as Files

Linux treats every hardware device as a file.

Examples:
• /dev/sda → Hard disk
• /dev/tty → Terminal
• /dev/usb → USB devices

Makes device management simple.

🗀 /proc – Process & Kernel Information

A virtual, read-only filesystem showing **live system info**.

Examples:
• /proc/cpuinfo
• /proc/meminfo
• /proc/1234 → Info about process 1234

Useful for debugging and monitoring.

🗀 /boot – Boot Loader & Kernel Files

Contains files required for system startup:

• Kernel (vmlinuz)
• GRUB configurations
• initramfs

Without /boot, Linux cannot boot.

🗀 /lib & /lib64 – Shared Libraries

Contains shared libraries required by commands in /bin and /sbin.

Equivalent to **DLL files in Windows**.